

Requirement Understanding Document

Project Title

Community Library Management System

Introduction

This project focuses on developing a console-based application to manage the day-to-day operations of a community library. The system will help library staff manage books, members, borrowing activities, returns, fines, and reports in an organized and efficient way.

The application will be developed using .NET 8, Entity Framework Core, and PostgreSQL following a layered architecture approach for better maintainability and scalability.

Project Objective

The main objective of this project is to create a simple and reliable library management solution that supports:

- Member management
- Book and copy management
- Borrowing and return operations
- Fine calculation and tracking
- Report generation

The application should enforce all business rules related to borrowing limits, member eligibility, fine restrictions, and book availability.

Architecture Overview

The application follows a 4-project layered architecture:

LibraryManagementModelLibrary

Contains:

- Entity models
- DTOs
- Enums

LibraryManagementDALLibrary

Responsible for:

- Database communication
- Entity Framework Core operations
- Repository implementations

LibraryManagementBLLibrary

Handles:

- Business rules
- Validation logic
- Transaction processing

LibraryManagementFEApplication

Provides:

- Console-based user interface
 - Menus
 - User interaction handling
-

Stakeholders

The primary users of the system are:

- Library staff managing books and members
 - Library administrators monitoring reports and fines
 - Developers maintaining or extending the application in future
-

Scope of the Project

Included Features

Member Management

- Add new members
- View all members
- Search members by phone number or email
- Update member status
- Deactivate members
- Assign membership types

Book Management

- Add books
- Add multiple copies for a book
- Manage book categories
- Search books by title, author, or category
- Mark copies as damaged or unavailable
- View available books

Borrowing and Return Management

- Borrow books for eligible members
- Return borrowed books
- Track due dates
- Prevent invalid borrowing operations
- Update copy availability automatically

Fine Management

- Calculate fines for late returns

- Handle damaged or lost book penalties
- Track pending and paid fines
- View fine payment history

Reports

- Currently borrowed books
- Overdue books
- Members with pending fines
- Most borrowed books
- Available books by category
- Member borrowing history

Database Support

- PostgreSQL database integration
- Entity Framework Core for ORM operations
- Stored functions for selected reporting and calculation operations

Features Not Included

The following features are outside the scope of this version:

- Web application
- Mobile application
- Authentication system
- Email or SMS notifications
- Barcode scanner hardware integration
- Online payment gateway integration

Functional Requirements

Member Management Requirements

The system should support:

- Adding members
- Viewing member details
- Searching members
- Updating membership status

- Deactivating accounts
 - Assigning membership types such as Basic, Student, and Premium
-

Book Management Requirements

The system should support:

- Adding books
 - Creating or selecting categories
 - Adding individual book copies using barcode numbers
 - Searching books using different criteria
 - Viewing available books
 - Updating copy status
-

Borrowing Requirements

Before borrowing a book, the system must validate:

- Member account is active
- Borrowing limit is not exceeded
- Member does not have excessive unpaid fines
- Book copy is available
- Same member has not already borrowed the same ISBN without returning it

If validation succeeds:

- Borrowing record should be created
 - Book copy status should become Borrowed
-

Return Requirements

When returning a book:

- Return date should be recorded
 - Borrowing status should be updated
 - Book copy should become available again
 - Fine should be calculated if overdue
-

Fine Management Requirements

The system should:

- Calculate unpaid fines
- Support different fine types
- Allow fine payments
- Display fine history

Supported fine types:

- Late return
 - Damaged book
 - Lost book
-

Business Rules

- Only active members can borrow books
 - Borrowing limits depend on membership type
 - Members with unpaid fines above ₹500 cannot borrow books
 - Only available copies can be borrowed
 - Damaged or lost books cannot be borrowed
 - Fine amount for late returns is ₹10 per delayed day
 - Lost books may require paying the full book price
-

Data Requirements

Main Entities

- MembershipType
 - BookCategory
 - Book
 - BookCopy
 - Member
 - Borrowing
 - Fine
 - FinePayment
-

Non-Functional Requirements

- The console interface should be easy to navigate
 - Proper validation messages should be displayed
 - The application should handle exceptions gracefully
 - Data should be stored reliably in PostgreSQL
 - Monetary values should use proper currency formatting
 - The project should support future enhancements with minimal restructuring
-

User Interface Requirements

The console application should provide:

- Clear menus
- Readable tabular outputs
- Simple navigation flow
- Meaningful prompts and error messages

The system should prefer user-friendly inputs wherever possible, such as category names instead of category IDs.

Technical Requirements

- .NET 8
 - PostgreSQL
 - Entity Framework Core
 - Console Application
 - Layered Architecture
 - Repository Pattern
 - EF Core Transactions
-

Success Criteria

The project will be considered successful if:

- Users can manage books and members without issues
 - Borrowing and return workflows operate correctly
 - Business rules are enforced consistently
 - Reports display accurate information
 - Data is stored and retrieved correctly from PostgreSQL
 - The application handles invalid input safely without crashing
-

Conclusion

The Community Library Management System is designed to simplify library operations using a modular and maintainable architecture. The project emphasizes proper database design, business rule validation, and clean separation of concerns while providing a straightforward console-based user experience.